

For the technically inclined, you may note that `my_bot` (which is the return value of `context.bot`) is an object of class `telegram.Bot`. Further, this object, i.e., `my_bot`, being an object of class `telegram.Bot`, has a method `send_message()`. So you can use this method to send a message to the bot user.

Note: You can confirm that the return of `context.bot` is actually an object of class `telegram.Bot` by looking at the source code for this in the file `callbackcontext.py`. You can find this file on GitHub at <https://github.com/python-telegram-bot/python-telegram-bot/blob/7b3b278c7c09d2961a696e58b955914486d96313/telegram/ext/callbackcontext.py>. You will find the definition of `bot()` around lines 155-157. Also the class `telegram.Bot` has a method `send_message()` by looking at the source code of the file `bot.py` available at <https://github.com/python-telegram-bot/python-telegram-bot/blob/master/telegram/bot.py>. You will get the code for the method `send_message()` around lines 265-275.

But in the above scheme, there is one problem — you need to register this *command*, i.e., `get()` in the dispatcher. To do this, the module has a class *Handler*, and from this class many other classes are inherited such as *CommandHandler*, *MessageHandler*, etc. Here, since `get()` is a command, we will use the specific *Handler* meant to handle commands, which is *Commandhandler*. Note that the class *Commandhandler* sub-classes the class *Handler*, i.e., it inherits from the class *Handler*. So the scheme works like this.

You first create an object of class *CommandHandler* (I have named it `start_handler`) and pass to it the name of the command as the first parameter and the name of the callback function as the second parameter. So the line of code will be as follows:

```
start_handler = CommandHandler('start',
start)
```

Note: You can see the source code for the *Dispatcher* class in the file `dispatcher.py` at <https://github.com/python-telegram-bot/python-telegram-bot/blob/7b3b278c7c09d2961a696e58b955914486d96313/telegram/ext/dispatcher.py>.

Having created an object of type *CommandHandler*, you can add it to the *Dispatcher* class object (which in our code is `my_dispatcher`), using the *method* `add_handler()` of the *Dispatcher* class. So, the code will be as follows:

```
1 from telegram.ext import Updater,
CommandHandler
2 my_updater = Updater(token='Your_token_
here', use_context=True)
3 my_dispatcher = my_updater.dispatcher
4
5 def start(update, context):
6 chat_id = update.effective_chat.id #
Get id of the chat.
7 f_name = update.effective_chat.first_
name # First name of bot user
8 l_name = update.effective_chat.last_
name # Last name of bot user
9 full_name = f_name + ' ' + l_name #
Full name of bot user
10 out_text = 'Hi {0} {1} from bot'.
format(f_name, l_name)
11 # The print functions are to show the
contents of the variables
12 # You can remove the print functions
13 print(update)
14 print('effective_chat ->', update.
effective_chat)
15 print('chat_id->', chat_id)
16 print('bot->', context.bot)
```

```
17
18 my_bot = context.bot # my_bot is
an object of type telegram.Bot
19 # print(my_bot)
20 my_bot.send_message(chat_id, text=
out_text)
21 start_handler =
CommandHandler('start', start)
22 my_dispatcher.add_handler(start_
handler)
23 # my_updater.start_polling() starts
the bot, and the bot begins
24 # to start polling Telegram for any
chat updates.
25 # The bot has its own separate
threads so it wont halt your Python
script.
26 my_updater.start_polling()
27 # my_updater.idle() command is used
to block the script until
28 # the user sends a command to break
from the Python script such as ctrl-c
29 on windows
30 my_updater.idle()
```

The complete code for this bot is available on GitHub at https://github.com/ag999git/jupyter_notebooks/blob/master/telegram-bot.

Further extensions

You can improvise and extend the functionality of the bot. This bot accepts only one command, namely, `/start`. You can create a message handler. For that you will need an object of class *MessageHandler*. You can also try using *Webhooks* or install your bot on a remote server like Heroku, PythonAnywhere, etc. 

References

- [1] Complete documentation of a Python Telegram bot is available at <https://readthedocs.org/projects/python-telegram-bot/downloads/pdf/stable/>
- [2] <https://python-telegram-bot.readthedocs.io/en/latest/telegram.ext.updater.html#telegram.ext.Updater>
- [3] <https://python-telegram-bot.readthedocs.io/en/latest/telegram.ext.dispatcher.html#telegram.ext.Dispatcher>

By: Anurag Gupta

The author is an Engineer by education but a police officer by profession. He is a Python enthusiast who has recently co-authored a book on Python named *Python Programming: Problem Solving, Packages and Libraries*